

Taller de Matemática

Análisis de tiempos de ejecución



Motivación

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

- ¿Cuántas veces imprime el valor de la variable i?

Motivación

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

- ¿Cuántas veces imprime el valor de la variable i?
- ¿Como se cuando finaliza la ejecución?

Motivación

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

- ¿Cuántas veces imprime el valor de la variable i?
- ¿Como se cuando finaliza la ejecución?
- ¿Es relevante el símbolo de igualdad?
¿Por qué? ¿Puedo quitarlo?

Motivación

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

- ¿Cuántas veces imprime el valor de la variable i?
- ¿Como se cuando finaliza la ejecución?
- ¿Es relevante el símbolo de igualdad?
¿Por qué? ¿Puedo quitarlo?

Resultado por consola:

0.0 0.25 0.5 0.75 1.0

Motivación

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

- ¿Cuántas veces imprime el valor de la variable *i*?
- ¿Como se cuando finaliza la ejecución?
- ¿Es relevante el símbolo de igualdad?
¿Por qué? ¿Puedo quitarlo?

Resultado por consola:

0.0 0.25 0.5 0.75 1.0

¿Como se expresa en matemática esa cantidad de iteraciones?

Motivación

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

- ¿Cuántas veces imprime el valor de la variable i?
- ¿Como se cuando finaliza la ejecución?
- ¿Es relevante el símbolo de igualdad?
¿Por qué? ¿Puedo quitarlo?

Resultado por consola:

0.0 0.25 0.5 0.75 1.0

¿Como se expresa en matemática esa cantidad de iteraciones?

Utilizando notación sigma: Sumatoria (suma repetida)

Motivación

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

- ¿Cuántas veces imprime el valor de la variable i?
- ¿Como se cuando finaliza la ejecución?
- ¿Es relevante el símbolo de igualdad?
¿Por qué? ¿Puedo quitarlo?

Resultado por consola:

0.0 0.25 0.5 0.75 1.0

¿Como se expresa en matemática esa cantidad de iteraciones?

Utilizando notación sigma: Sumatoria (suma repetida)

Notacion Sigma:

$$\sum_{i=1}^n f(i)$$

donde i toma valores enteros (1, 2, 3, ..., n)
Los extremos también se cuentan (tanto i=1 como i=n)

En nuestro caso, se restringe a valores naturales

Notacion Sigma: Σ

Diagram illustrating the components of the sigma notation $\sum_{i=1}^n f(i)$:

- Índice final**: Points to n .
- Función que depende del índice**: Points to $f(i)$.
- Índice de suma**: Points to i .
- Índice inicial**: Points to 1 .

The notation is $\sum_{i=1}^n f(i)$.

Si $f(i)$ es una constante: $f(i) = C$

Tomando $n = 5$, como el ejemplo previo

Notacion Sigma: Σ

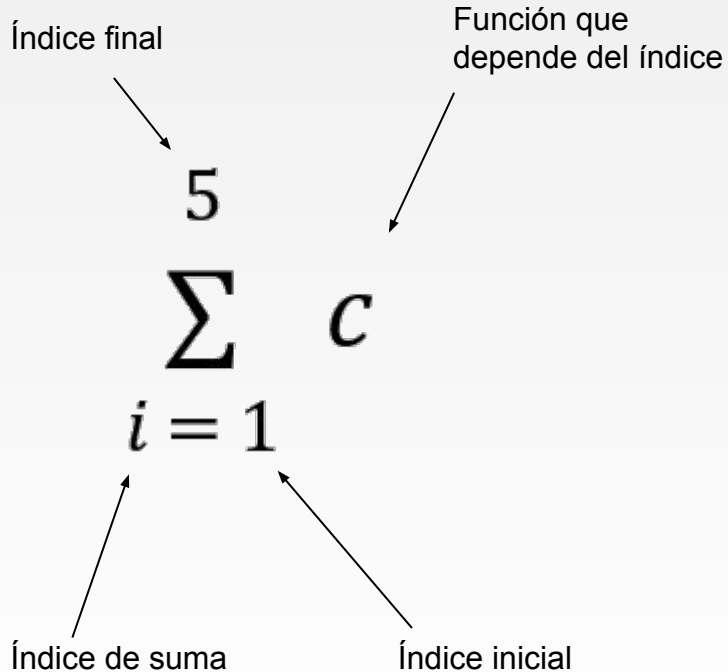


Diagram illustrating the components of the sigma notation $\sum_{i=1}^5 c$:

- Índice final**: Points to the number 5.
- Función que depende del índice**: Points to the variable c .
- Índice de suma**: Points to the variable i .
- Índice inicial**: Points to the number 1.

The notation is: $\sum_{i=1}^5 c$

Si $f(i)$ es una constante: $f(i) = C$

Tomando $n = 5$, como el ejemplo previo

La sumatoria se lee: “Sumá el número C , comenzando desde $i=1$ hasta $i=5$ ”

Notacion Sigma: Σ

Diagram illustrating the components of the sigma notation $\sum_{i=1}^5 c$:

- Índice final: 5
- Función que depende del índice: c
- Índice de suma: i
- Índice inicial: 1

Si $f(i)$ es una constante: $f(i) = C$

Tomando $n = 5$, como el ejemplo previo

La sumatoria se lee: “Sumá el número C , comenzando desde $i=1$ hasta $i=5$ ”

desde $i = 1$ hasta $i = 5$:

C

$i = 1$

Notacion Sigma: Σ

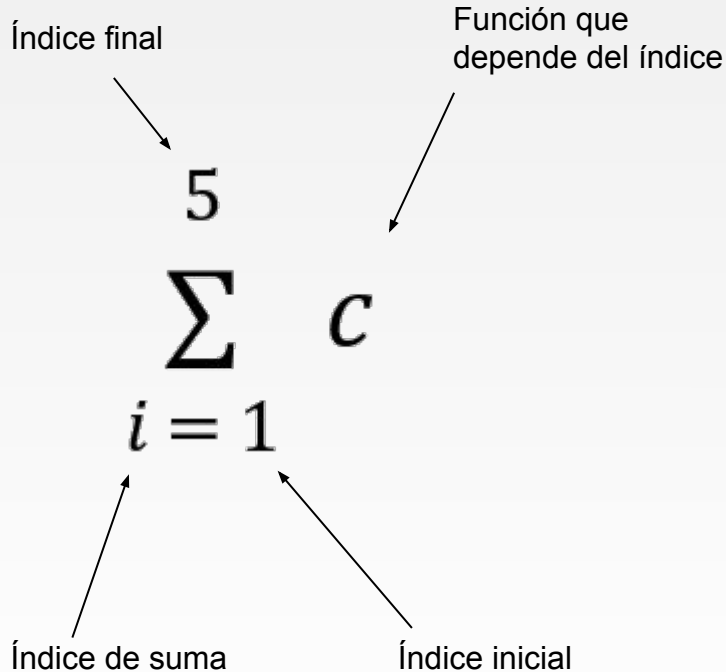


Diagram illustrating the components of the sigma notation $\sum_{i=1}^5 c$:

- Índice final**: Points to the number 5.
- Función que depende del índice**: Points to the variable c .
- Índice de suma**: Points to the variable i .
- Índice inicial**: Points to the number 1.

The notation is: $\sum_{i=1}^5 c$

Si $f(i)$ es una constante: $f(i) = C$

Tomando $n = 5$, como el ejemplo previo

La sumatoria se lee: “Sumá el número C , comenzando desde $i=1$ hasta $i=5$ ”

desde $i = 1$ hasta $i = 5$:

$$C + C$$

$$i = 1$$

Notacion Sigma: Σ

Diagram illustrating the components of the sigma notation $\sum_{i=1}^5 c$:

- Índice final: 5
- Función que depende del índice: c
- Índice de suma: i
- Índice inicial: 1

Si $f(i)$ es una constante: $f(i) = C$

Tomando $n = 5$, como el ejemplo previo

La sumatoria se lee: “Sumá el número C , comenzando desde $i=1$ hasta $i=5$ ”

desde $i = 1$ hasta $i = 5$:

$$C + C + C$$

$$i = 1$$

Notacion Sigma: Σ

Diagram illustrating the components of the sigma notation $\sum_{i=1}^5 c$:

- Índice final: 5
- Función que depende del índice: c
- Índice de suma: i
- Índice inicial: 1

Si $f(i)$ es una constante: $f(i) = C$

Tomando $n = 5$, como el ejemplo previo

La sumatoria se lee: “Sumá el número C , comenzando desde $i=1$ hasta $i=5$ ”

desde $i = 1$ hasta $i = 5$:

$$\begin{array}{ccccccc} C & + & C & + & C & + & C & + & C \\ i = 1 & & & & \dots & & & & i = 5 \end{array}$$

Notacion Sigma: Σ

Diagram illustrating the components of the sigma notation $\sum_{i=1}^5 c$:

- Índice final: 5
- Función que depende del índice: c
- Índice de suma: i
- Índice inicial: 1

Si $f(i)$ es una constante: $f(i) = C$

Tomando $n = 5$, como el ejemplo previo

La sumatoria se lee: “Sumá el número C , comenzando desde $i=1$ hasta $i=5$ ”

desde $i = 1$ hasta $i = 5$:

$$\begin{array}{ccccccc} C & + & C & + & C & + & C & + & C \\ i = 1 & & & & \dots & & & & i = 5 \end{array}$$

Resultado: $5 * C$

Notacion Sigma: Σ

Diagram illustrating the components of the sigma notation $\sum_{i=1}^n f(i)$:

- n : Índice final (Final index)
- $f(i)$: Función que depende del índice (Function that depends on the index)
- i : Índice de suma (Index of summation)
- 1 : Índice inicial (Initial index)

Si $f(i)$ es una constante: $f(i) = C$

Fórmula de sumatoria de una constante

$$\sum_{i=a}^b c = c^* (b - a + 1)$$

Notacion Sigma: Σ

Diagram illustrating the components of the sigma notation $\sum_{i=1}^n f(i)$:

- Índice final: n
- Función que depende del índice: $f(i)$
- Índice de suma: i
- Índice inicial: 1

Si $f(i)$ es una constante: $f(i) = C$

Fórmula de sumatoria de una constante

$$\sum_{i=a}^b C = C^* (b - a + 1)$$

Con $a = 1$; $b = 5$:

Notacion Sigma: Σ

Diagram illustrating the components of the sigma notation $\sum_{i=1}^n f(i)$:

- Índice final**: Points to n .
- Función que depende del índice**: Points to $f(i)$.
- Índice de suma**: Points to i .
- Índice inicial**: Points to 1 .

Si $f(i)$ es una constante: $f(i) = C$

Fórmula de sumatoria de una constante

$$\sum_{i=a}^b c = c * (b - a + 1)$$

Con $a = 1$; $b = 5$: $c * (5 - 1 + 1) = 5C$

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 0.5$

¿Cuántos decimales tengo?

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 0.5$

¿Cuántos decimales tengo?

Observar la cantidad de números luego de la coma. En este caso:

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 0.5$

¿Cuántos decimales tengo?

Observar la cantidad de números luego de la coma. En este caso:

un único decimal, entonces $d = 1$

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 0.5$

¿Cuántos decimales tengo?

Observar la cantidad de números luego de la coma. En este caso:

un único decimal, entonces $d = 1$

$$0.5 \cdot 10^1 / 10^1 = 5 / 10$$

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 0.5$

¿Cuántos decimales tengo?

Observar la cantidad de números luego de la coma. En este caso:

un único decimal, entonces $d = 1$

$$0.5 \cdot 10^1 / 10^1 = 5 / 10$$

Simplificamos, buscando un máximo común divisor (MCD). $\text{MCD} = 5$

$$1 / 2$$

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 3.25$

¿Cuántos decimales tengo?

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 3.25$

¿Cuántos decimales tengo?

Observar la cantidad de números luego de la coma. En este caso:

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 3.25$

¿Cuántos decimales tengo?

Observar la cantidad de números luego de la coma. En este caso:

dos decimales, entonces $d = 2$

$$3.25 \cdot 10^2 / 10^2 = 325 / 100$$

Convertir un numero decimal a fraccionario

Pasos para obtener la expresión fraccionaria de un número decimal exacto:

Sea n , nuestro número decimal exacto.

1. Contabilizar la cantidad de decimales (dígitos luego de la coma). Llamemos a ese número d .
2. Multiplicar y dividir el número original $\cdot 10^d$
3. Simplificar la expresión, buscando un MCD.

Por ejemplo: Sea $n = 3.25$

¿Cuántos decimales tengo?

Observar la cantidad de números luego de la coma. En este caso:

dos decimales, entonces $d = 2$

$$3.25 \cdot 10^2 / 10^2 = 325 / 100$$

Simplificamos, buscando un máximo común divisor (MCD). $\text{MCD} = 25$

$$13 / 4$$

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```


Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} > 1.0$, con un incremento de $1/4$

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} > 1.0$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} > 1.0$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$[(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] + 1 =$$

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} > 1.0$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$[(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] + 1 =$$
$$[(1.0 - 0.0) / 1/4] + 1 =$$

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} > 1.0$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(1.0 - 0.0) / 1/4] + 1 = \\ & [(1.0 - 0.0) * 4] + 1 = \end{aligned}$$

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} > 1.0$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(1.0 - 0.0) / 1/4] + 1 = \\ & [(1.0 - 0.0) * 4] + 1 = 4 + 1 = 5 \end{aligned}$$

Volviendo al ejemplo del inicio de la clase

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (double i = 0.0; i <= 1.0; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} > 1.0$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(1.0 - 0.0) / 1/4] + 1 = \\ & [(1.0 - 0.0) * 4] + 1 = 4 + 1 = 5 \end{aligned}$$

Observación: Sumarle uno al final, debido a que posee un igual en la condición de terminación del bucle.

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

0.25: $0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

0.25: $0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

0.25: $0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$[(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] =$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] = \\ [(n/4 - 0.0) / 1/4] = \end{aligned}$$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] &= \\ [(n/4 - 0.0) / 1/4] &= \\ [(n/4 - 0.0) * 4] &= \end{aligned}$$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] &= \\ [(n/4 - 0.0) / 1/4] &= \\ [(n/4 - 0.0) * 4] &= n/4 * 4 = n \end{aligned}$$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] &= \\ [(n/4 - 0.0) / 1/4] &= \\ [(n/4 - 0.0) * 4] &= n/4 * 4 = n \end{aligned}$$

Expresado en notación sigma:

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] &= \\ [(n/4 - 0.0) / 1/4] &= \\ [(n/4 - 0.0) * 4] &= n/4 * 4 = n \end{aligned}$$

Expresado en notación sigma:

$$\sum_{i=0}^n c \quad \text{¿Es correcto?}$$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] &= \\ [(n/4 - 0.0) / 1/4] &= \\ [(n/4 - 0.0) * 4] &= n/4 * 4 = n \end{aligned}$$

Expresado en notación sigma:

$$\sum_{i=0}^n c \quad \text{¿Es correcto? NO!}$$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] &= \\ [(n/4 - 0.0) / 1/4] &= \\ [(n/4 - 0.0) * 4] &= n/4 * 4 = n \end{aligned}$$

Expresado en notación sigma:

$$\sum_{i=0}^{n-1} c$$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

$$0.25: 0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] &= \\ [(n/4 - 0.0) / 1/4] &= \\ [(n/4 - 0.0) * 4] &= n/4 * 4 = n \end{aligned}$$

Expresado en notación sigma:

$$\sum_{i=0}^{n-1} c = \sum_{i=1}^n c$$

Ejemplo 1: For

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+= 0.25) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Depende de la variable n. Mientras se cumpla la condición: $i < n/4$

Convertir expresión decimal a fraccionaria:

0.25: $0.25 * 10^2 / 10^2 = 25 / 100 = 1 / 4$

El for comienza en $i_{\text{inicial}} = 0.0$ y termina en $i_{\text{final}} = n/4$, con un incremento de $1/4$

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} [(i_{\text{final}} - i_{\text{inicial}}) / \text{incremento}] &= \\ [(n/4 - 0.0) / 1/4] &= \\ [(n/4 - 0.0) * 4] &= n/4 * 4 = n \end{aligned}$$

Expresado en notación sigma:

$$\sum_{i=0}^{n-1} c = \sum_{i=1}^n c = c * (n - 1 + 1) = c * n$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+=0.25) {  
        for (int j = n/4; j <= n/2; j++) {  
            System.out.println(i);  
        }  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
    [REDACTED]  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
    [REDACTED]  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n . Mientras se cumpla la condición: $j < n/2$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n . Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n . Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n . Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$[(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 =$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n. Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$[(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 =$$
$$[(n/2 - n/4) / 1] + 1 =$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n . Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(n/2 - n/4) / 1] + 1 = 2n/4 - n/4 + 1 = \end{aligned}$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n . Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(n/2 - n/4) / 1] + 1 = 2n/4 - n/4 + 1 = n/4 + 1 \end{aligned}$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n . Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(n/2 - n/4) / 1] + 1 = 2n/4 - n/4 + 1 = n/4 + 1 \end{aligned}$$

Expresado en notación sigma:

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n. Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(n/2 - n/4) / 1] + 1 = 2n/4 - n/4 + 1 = n/4 + 1 \end{aligned}$$

Expresado en notación sigma:

$$\begin{aligned} & \frac{n}{4} + 1 \\ & \sum_{i=1} c = \end{aligned}$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    [REDACTED]  
    for (int j = n/4; j <= n/2; j++) {  
        System.out.println(i);  
    }  
    [REDACTED]  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n. Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(n/2 - n/4) / 1] + 1 = 2n/4 - n/4 + 1 = n/4 + 1 \end{aligned}$$

Expresado en notación sigma:

$$\begin{aligned} & \frac{n}{4} + 1 \\ & \sum_{i=1}^C C = C * (n/4 + 1) \end{aligned}$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+=0.25) {  
        for (int j = n/4; j <= n/2; j++) {  
            System.out.println(i);  
        }  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n. Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(n/2 - n/4) / 1] + 1 = 2n/4 - n/4 + 1 = n/4 + 1 \end{aligned}$$

Expresado en notación sigma:

$$\sum_{i=0}^{\frac{n}{4} + 1} c =$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+=0.25) {  
        for (int j = n/4; j <= n/2; j++) {  
            System.out.println(i);  
        }  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n . Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(n/2 - n/4) / 1] + 1 = 2n/4 - n/4 + 1 = n/4 + 1 \end{aligned}$$

Expresado en notación sigma:

$$\sum_{i=1}^n \sum_{j=1}^{\frac{n}{4} + 1} c =$$

Ejemplo 2: For anidado

Suponiendo el siguiente código escrito en Java:

```
public static void example(double n) {  
    for (double i = 0.0; i < n/4; i+=0.25) {  
        for (int j = n/4; j <= n/2; j++) {  
            System.out.println(i);  
        }  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for más interno?

Depende de la variable n. Mientras se cumpla la condición: $j < n/2$

El for comienza en $j_{\text{inicial}} = n/4$ y termina en $j_{\text{final}} > n/2$, con un incremento de 1

La cantidad de iteraciones puede calcularse como:

$$\begin{aligned} & [(j_{\text{final}} - j_{\text{inicial}}) / \text{incremento}] + 1 = \\ & [(n/2 - n/4) / 1] + 1 = 2n/4 - n/4 + 1 = n/4 + 1 \end{aligned}$$

Expresado en notación sigma:

$$\sum_{i=1}^n \sum_{i=1}^{\frac{n}{4} + 1} C = n * [(n/4 + 1) * C]$$

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 1$ hasta 20, solo las potencias

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 1$ hasta 20, solo las potencias

Iteraciones de i :

$i = 1$ $i = 2$ $i = 4$ $i = 8$ $i = 16$

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Desde $i = 1$ hasta 20, solo las potencias

Iteraciones de i :

$i = 1$ $i = 2$ $i = 4$ $i = 8$ $i = 16$

Total de 5 iteraciones

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Desde $i = 1$ hasta 20, solo las potencias

Iteraciones de i :

$i = 1$ $i = 2$ $i = 4$ $i = 8$ $i = 16$

Total de 5 iteraciones

El for comienza en $i_{\text{inicial}} = 1$ y termina en $i_{\text{final}} = 20$, con un incremento multiplicativo = 2

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 1$ hasta 20, solo las potencias

Iteraciones de i :

$i = 1$ $i = 2$ $i = 4$ $i = 8$ $i = 16$

Total de 5 iteraciones

El for comienza en $i_{\text{inicial}} = 1$ y termina en $i_{\text{final}} = 20$, con un incremento multiplicativo = 2

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{incremento}} \left(\frac{i_{\text{final}}}{i_{\text{inicial}}} \right) + 1$$

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 1$ hasta 20, solo las potencias

Iteraciones de i :

$i = 1$ $i = 2$ $i = 4$ $i = 8$ $i = 16$

Total de 5 iteraciones

El for comienza en $i_{\text{inicial}} = 1$ y termina en $i_{\text{final}} = 20$, con un incremento multiplicativo = 2

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{incremento}} \left(\frac{i_{\text{final}}}{i_{\text{inicial}}} \right) + 1$$

Se agrega el (+1) por la condición de terminación del bucle que toma el valor final.

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 1$ hasta 20, solo las potencias

Iteraciones de i :

$i = 1$ $i = 2$ $i = 4$ $i = 8$ $i = 16$

Total de 5 iteraciones

El for comienza en $i_{\text{inicial}} = 1$ y termina en $i_{\text{final}} = 20$, con un incremento multiplicativo = 2

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{incremento}} \left(\frac{i_{\text{final}}}{i_{\text{inicial}}} \right) + 1$$

Se agrega el (+1) por la condición de terminación del bucle que toma el valor final.

$$\log_2(20) + 1 = 4 + 1 = 5$$

Ejemplo 3: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 1; i <= 20; i*= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 1$ hasta 20, solo las potencias

Iteraciones de i :

$i = 1$ $i = 2$ $i = 4$ $i = 8$ $i = 16$

Total de 5 iteraciones

El for comienza en $i_{\text{inicial}} = 1$ y termina en $i_{\text{final}} = 20$, con un incremento multiplicativo = 2

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{incremento}} \left(\frac{i_{\text{final}}}{i_{\text{inicial}}} \right) + 1$$

Se agrega el (+1) por la condición de terminación del bucle que toma el valor final.

$$\log_2(20) + 1 = 4 + 1 = 5$$

El resultado del logaritmo es solo la parte entera del número.

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde i = 20 hasta 1 (sin incluirlo)

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde i = 20 hasta 1 (sin incluirlo)

Iteraciones de i:

i = 20 i = 10 i = 5 i = 2

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde i = 20 hasta 1 (sin incluirlo)

Iteraciones de i:

i = 20 i = 10 i = 5 i = 2

Total de 4 iteraciones

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

El for comienza en $i_{\text{inicial}} = 20$ y termina en $i_{\text{final}} = 1$, con un decremento exponencial $= 2$

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 20$ hasta 1 (sin incluirlo)

Iteraciones de i :

$i = 20$ $i = 10$ $i = 5$ $i = 2$

Total de 4 iteraciones

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 20$ hasta 1 (sin incluirlo)

Iteraciones de i :

$i = 20$ $i = 10$ $i = 5$ $i = 2$

Total de 4 iteraciones

El for comienza en $i_{\text{inicial}} = 20$ y termina en $i_{\text{final}} = 1$, con un decremento exponencial $= 2$

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{decremento}} \left(\frac{i_{\text{final}}}{i_{\text{inicial}}} \right)$$

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde i = 20 hasta 1 (sin incluirlo)

Iteraciones de i:

i = 20 i = 10 i = 5 i = 2

Total de 4 iteraciones

El for comienza en i_inicial = 20 y termina en i_final = 1, con un decremento exponencial = 2

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{decremento}} \left(\frac{i_{\text{final}}}{i_{\text{inicial}}} \right)$$

¿Es correcto?

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde i = 20 hasta 1 (sin incluirlo)

Iteraciones de i:

i = 20 i = 10 i = 5 i = 2

Total de 4 iteraciones

El for comienza en i_inicial = 20 y termina en i_final = 1, con un decremento exponencial = 2

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{decremento}} \left(\frac{i_{\text{final}}}{i_{\text{inicial}}} \right)$$

¿Es correcto? **NO!**

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde $i = 20$ hasta 1 (sin incluirlo)

Iteraciones de i :

$i = 20$ $i = 10$ $i = 5$ $i = 2$

Total de 4 iteraciones

El for comienza en $i_{\text{inicial}} = 20$ y termina en $i_{\text{final}} = 1$, con un decremento exponencial $= 2$

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{decremento}} \left(\frac{i_{\text{inicial}}}{i_{\text{final}}} \right)$$

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Desde $i = 20$ hasta 1 (sin incluirlo)

Iteraciones de i :

$i = 20$ $i = 10$ $i = 5$ $i = 2$

Total de 4 iteraciones

El for comienza en $i_{\text{inicial}} = 20$ y termina en $i_{\text{final}} = 1$, con un decremento exponencial $= 2$

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{decremento}} \left(\frac{i_{\text{inicial}}}{i_{\text{final}}} \right)$$

En este caso, la condición de terminación del bucle no toma el valor final.

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?

Desde $i = 20$ hasta 1 (sin incluirlo)

Iteraciones de i :

$i = 20$ $i = 10$ $i = 5$ $i = 2$

Total de 4 iteraciones

El for comienza en $i_{\text{inicial}} = 20$ y termina en $i_{\text{final}} = 1$, con un decremento exponencial $= 2$

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{decremento}} \left(\frac{i_{\text{inicial}}}{i_{\text{final}}} \right)$$

En este caso, la condición de terminación del bucle no toma el valor final.

$$\log_2(20) = 4$$

Ejemplo 4: For

Suponiendo el siguiente código escrito en Java:

```
public static void main(String[] args) {  
    for (int i = 20; i > 1; i /= 2) {  
        System.out.println(i);  
    }  
}
```

¿Cuántas veces se ejecuta el bloque for?
Desde i = 20 hasta 1 (sin incluirlo)

Iteraciones de i:

i = 20 i = 10 i = 5 i = 2

Total de 4 iteraciones

El for comienza en i_inicial = 20 y termina en i_final = 1, con un decremento exponencial = 2

La cantidad de iteraciones puede calcularse como:

$$\log_{\text{decremento}} \left(\frac{i_{\text{inicial}}}{i_{\text{final}}} \right)$$

En este caso, la condición de terminación del bucle no toma el valor final.

$$\log_2(20) = 4$$

El resultado del logaritmo es solo la parte entera del número.